DATOS PARA LA TABLA DE COMPLEJIDADES

# Y NOMBRE DEL PROBLEMA	Estructura Principal	Complejidad Temporal	Complejidad Espacial	Justificación
#1 Two Sum	dict (Tabla Hash / Diccionario)	O(n)	O(n)	Se recorre el arreglo de tamaño n una sola vez en O(n), en cada iteración se busca el complemento en el diccionario en tiempo constante O(1) y se inserta el elemento actual, almacenando hasta n elementos.
#238 Product of Array Except Self	Listas / Arreglos (list)	O(n)	O(1) auxiliar, O(n) contando la lista de salida	Se realizan dos recorridos lineales independientes sobre la lista O(n) para acumular productos a la izquierda y derecha, no se utiliza división y el espacio auxiliar es O(1) sin contar la lista del resultado.
#56 Merge Intervals	Listas / Arreglos (list)	O(n log n)	O(n)	Se ordenan los n intervalos según su punto de inicio en O(n log n), después, se hace un recorrido lineal O(n) donde se comparan los límites para fusionar los rangos que se traslapan.

DATOS PARA LA TABLA DE COMPLEJIDADES

# Y NOMBRE DEL PROBLEMA	Estructura Principal	Complejidad Temporal	Complejidad espacial	Justificación
#929 Unique Email Addresses	set (Conjunto) y Expresiones Regulares	O(N*M)	O(N*M)	Se procesan N correos de longitud máxima M re.sub() limpia las cadenas en O(M) y los correos únicos se guardan en un conjunto en O(N*M)
#592 Fraction Addition and Subtraction	Expresiones Regulares y Listas	O(N)	O(N)	re.findall() extrae los pares numerador/denominador en O(N). Se opera y simplifica cada fracción con math.gcd en tiempo logarítmico, resultando en O(N) global.
#65 Valid Number	Expresiones Regulares	O(N)	O(1)	re.fullmatch() evalúa la cadena mediante un autómata finito determinista que recorre la longitud N una sola vez usando memoria constante.

DATOS PARA LA TABLA DE COMPLEJIDADES

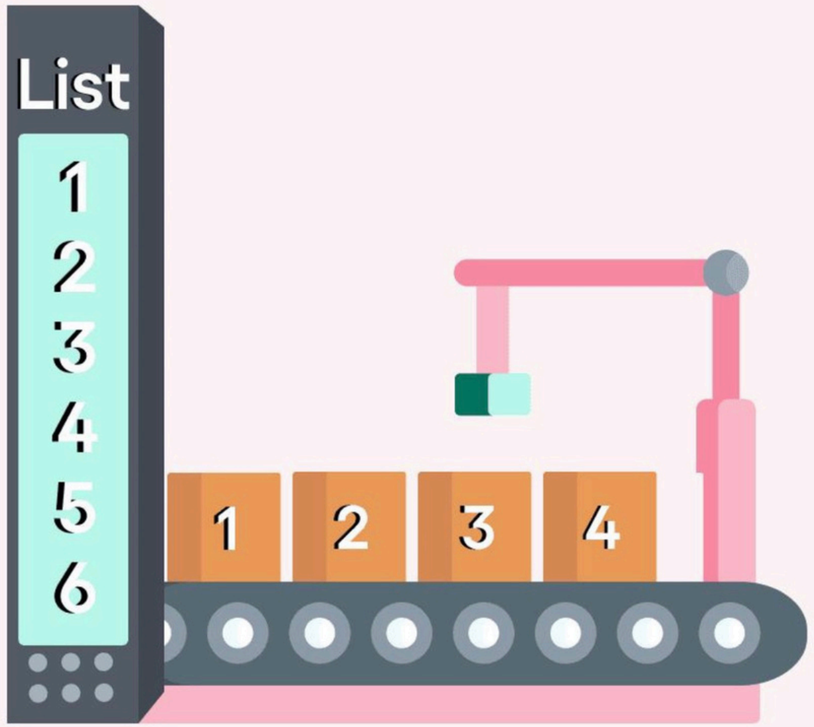
# Y NOMBRE DEL PROBLEMA	Estructura Principal	Complejidad Temporal	Complejidad espacial	Justificación
#20 Valid Parentheses	Lista de Python usada como pila	O(N)	O(N)	Se recorre la cadena una sola vez, en el peor caso, la pila puede guardar hasta caracteres.
#150 Evaluate Reverse Polish Notation	Lista de Python usada como pila	O(N)	O(N)	Se recorren los tokens una vez y se apilan los operandos, en el peor caso, la pila puede almacenar hasta elementos.
#739 Daily Temperatures	Lista de Python usada como pila monótona	O(N)	O(N)	Cada índice se agrega y se elimina de la pila como máximo una vez, la pila y la lista de resultados pueden ocupar hasta posiciones.

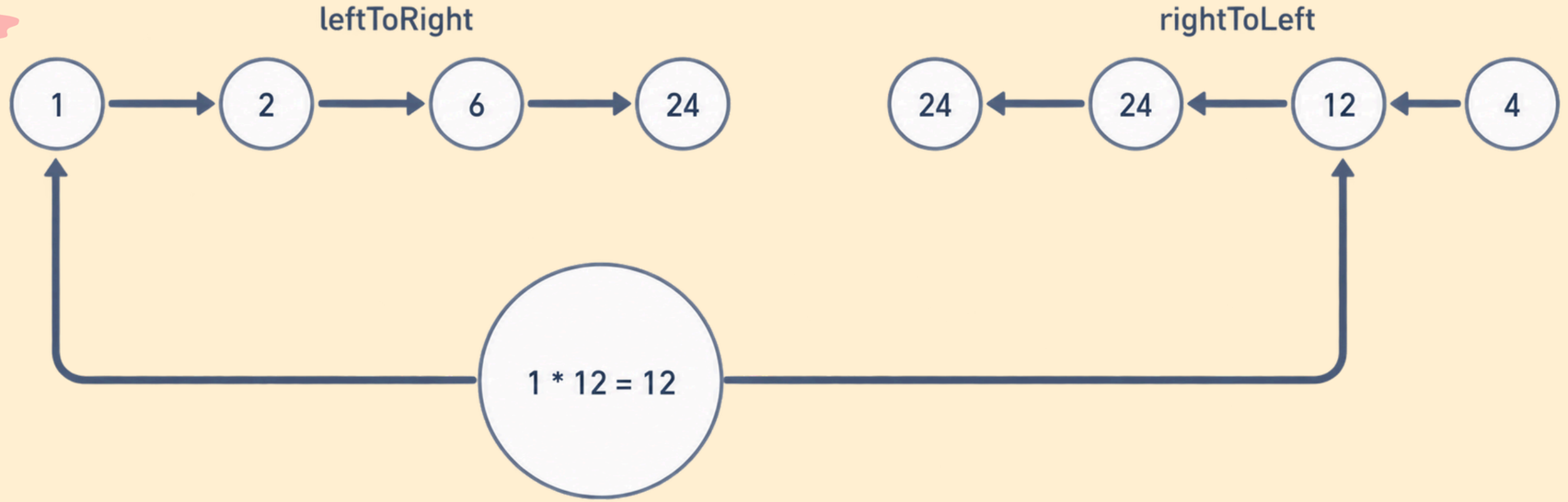
DATOS PARA LA TABLA DE COMPLEJIDADES

# Y NOMBRE DEL PROBLEMA	Estructura Principal	Complejidad Temporal	Complejidad espacial	Justificación
#232 Implement Queue using Stacks	Dos pilas (listas de Python).	O(1) amortizada	O(n)	Cada elemento se inserta en la pila de entrada y, como máximo, se transfiere una vez a la pila de salida durante toda su vida útil en la estructura, por lo que el tiempo por operación en promedio es constante. El espacio total es proporcional al número de elementos n almacenados en ambas pilas.
#622 Design Circular Queue	Arreglo estático (lista de tamaño fijo) con apuntadores (head y tail).	O(1) para todas las operaciones (enQueue, deQueue, Front, Rear, isEmpty, isFull).	O(k)	Todas las operaciones de inserción, eliminación y consulta utilizan índices directos y aritmética modular en tiempo constante. El espacio auxiliar está estrictamente determinado por la capacidad fija k de la cola.
#239 Sliding Window Maximum	Cola doblemente terminada (collections.deque) monótona.	O(n)	O(k)	Cada elemento de la lista se procesa, se añade y se elimina de la deque como máximo una vez, lo que garantiza un recorrido lineal en tiempo O(n). El espacio máximo en la estructura auxiliar está limitado por el tamaño de la ventana k..

Análisis y reflexión

En esta tarea aprendimos que la forma en que guardamos los datos influye en qué tan rápido funciona un programa, por ejemplo cuando usamos list.pop(0) se elimina el primer elemento de la lista pero los demás tienen que recorrerse para ocupar su lugar, por eso su complejidad es O(n), mientras que con deque.popleft() se puede quitar el primer elemento sin mover todos los demás y su complejidad es O(1), esto pasa porque las listas guardan sus elementos en posiciones consecutivas de memoria y al quitar el primero se tienen que acomodar los demás, mientras que una deque está diseñada para agregar y quitar elementos por los extremos sin hacer esos desplazamientos, por eso es importante elegir la estructura correcta dependiendo de lo que necesitemos hacer.





Uno de los problemas que más trabajo nos costó fue el #238 Product of Array Except Self porque no podíamos usar la división y tuvimos que buscar otra forma de calcular los productos, la solución fue hacer dos recorridos, uno de izquierda a derecha y otro de derecha a izquierda, guardando los resultados en la misma lista de respuesta para no crear otras listas auxiliares, primero guardamos los productos de los elementos que están a la izquierda y después los multiplicamos por los productos de la derecha, así conseguimos el resultado sin utilizar la división, con esto usamos un espacio auxiliar de O(1), aunque la lista de salida ocupa O(n), y entendimos que elegir una buena estrategia puede ayudar a resolver un problema usando menos memoria.

También aprendimos que re.fullmatch() sirve para validar una cadena completa, mientras que re.match() solo revisa desde el inicio. Por ejemplo, si usamos el patrón \d+ con la cadena "123abc", re.match() encuentra "123" y devuelve una coincidencia porque empieza con números, pero re.fullmatch() no encuentra ninguna porque toda la cadena no está formada únicamente por números. Esto es importante cuando queremos validar datos y asegurarnos de que cumplan con el formato completo, ya que no basta con que solo una parte sea correcta. En general, esta actividad nos ayudó a comprender mejor las estructuras de datos y las expresiones regulares, además de aprender que no solo importa que el código funcione, sino también entender cómo lo hace y qué tan eficiente es.

